Decidabilitate - Soluții

2023

1 Stop5

Se termină un program arbitrar P pe intrarea 5?

$$Stop5(Q) = \begin{cases} 1 & \text{dacă programul Q se termină pe intrarea 5} \\ 0 & \text{altfel} \end{cases}$$

Obiectiv 1: Arătăm că *Stop5* este *RE*.
Vom construi un program care întoarce în timp finit 1 dacă $Stop5(Q) = 1$.

```
1: function SolveStop5(Q):
2:     Q(5);
3:     return 1;
4: end function
```

Dacă apelul $Q(5)$ se termină, atunci programul se termină în urma apelului de return (linia 3).

$$SolveStop5(Q) = \begin{cases} 1 & \text{dacă Q(5) se termină} \\ \perp & \text{altfel} \end{cases}$$

Obiectiv 2: Arătăm că *Stop5* este $RE \setminus R$.

Pentru a îndeplini acest obiect vom face o reducere de la o problemă despre care știm că este $RE \setminus R$, și anume problema opririi ($PO <_T Stop5$). Pentru a reduce *PO* la *Stop5* avem nevoie de o funcție F **calculabilă** care să asocieze fiecărei intrări posibile a problemei *PO* o intrare convenabilă a problemei *Stop5*.

$$F : I_{PO} \to I_{Stop5}$$

PO primește la intrare un program P și o intrare w. *Stop5* primește la intrare un program Q.

$$F(P, w) = Q$$

Vom încerca să construim un program Q care să conțină un apel $P(w)$.

```
1: function Q(x):
2:     P(w)
3:     return 1;
4: end function
```

Am construit un program Q care ignoră intrarea și apelează $P(w)$. Pentru a valida reducerea trebuie să demonstrăm dubla implicație:

1. $PO(P, w) = 1 \to Stop5(Q) = 1$.
   - Dacă P(w) se oprește atunci și Q se oprește pe orice intrare (implicit, și pe intrarea 5).
2. $Stop5(Q) = 1 \to PO(P, w) = 1$.
   - Dacă $Q(5)$ se oprește atunci execuția codului a trecut de apelul $P(w)$.

2 StopOnly5

Se termină un program arbitrar **doar** pe intrarea 5?

Spre deosebire de problema anterioară, intuim că problema nu este nici măcar RE deoarece trebuie să validăm că proprietatea (neterminarea programului) trebuie testată pentru o infinitate de intrări.
Pentru a demonstra acest lucru, vom face o reducere de la o problemă despre care știm că nu este RE ($\overline{PO}$).

Obiectiv: $\overline{PO} <_T StopOnly5$

Similar, trebuie să găsim o funcție F calculabilă care construiește o intrare convenabilă pentru problema $StopOnly5$, pornind de la o intrare a problemei $\overline{PO}$.

$$F(P, w) = Q$$

```
1: function Q(x):
2:     if x == 5 then
3:         return
4:     else
5:         P(w);
6:         return
7:     end if
8: end function
```

Pentru a valida reducerea trebuie să demonstrăm dubla implicație:

1. $\overline{PO}(P, w) = 1 \rightarrow StopOnly5(Q) = 1$.
   - Dacă P(w) nu se oprește atunci nici Q nu se oprește pentru o intrare diferită de 5.
2. $StopOnly5(Q) = 1 \rightarrow \overline{PO}(P, w) = 1$.
   - Dacă Q se oprește doar pentru $x == 5$, atunci execuția codului nu trece niciodată de apelul $P(w)$ pentru alte intrări.

3 Exp

Calculează un program arbitrar 2^n pentru orice intrare $n \in \mathbb{N}$?

Pentru acest exercițiu vom considera cunoscut programul (menționat la curs) $runSteps(P, w, steps)$ care simulează primele *steps* operații elementare corespunzătoare programului P. Prin convenție, dacă P se termină în cel mult *steps* pași, *runSteps* întoarce valoarea returnată, altfel întoarce o referință nulă. Astfel, un apel al acestui program se termină întotdeauna.

Obiectiv: $\overline{PO} <_T Exp$

$$F(P, w) = Q$$

```
1: function Q(x):
2:     if runSteps(P, w, x) ≠ null then
3:         while(1);
4:     else
5:         return pow(2, x);
6:     end if
7: end function
```

Pentru a valida reducerea trebuie să demonstrăm dubla implicație:

1. $\overline{PO}(P, w) = 1 \rightarrow Exp(Q) = 1$.
   - Dacă P(w) nu se oprește pentru nicio intrare, atunci nu putem ajunge la linia 3. Programul se va termina întotdeauna cu ieșirea dorită.
2. $Exp(Q) = 1 \rightarrow \overline{PO}(P, w) = 1$.
   - Dacă programul întoarce 2^n pentru orice intrare validă x înseamnă că execuția codului nu ajunge la linia 3. Prin urmare, apelul P(w) nu se termină indiferent de valoarea lui x.

4 StopOn3

Se oprește un program arbitrar *P'* fix pe 3 intrari?

Obiectiv: $\overline{PO} <_T StopOn3$

$$F(P, w) = Q$$

```
1: function Q(x):
2:     if 0 ≤ x AND x < 3 then
3:         return
4:     else
5:         P(w)
6:     end if
7: end function
```

1. $\overline{PO}(P, w) = 1 \rightarrow StopOn3(Q) = 1$.
   - Dacă P(w) nu se oprește pentru nicio intrare, atunci nici Q nu se va opri pentru intrări diferite de 0, 1 și 2.
2. $StopOn3(Q) = 1 \rightarrow \overline{PO}(P, w) = 1$.
   - Din ipoteză, știm că StopOn3 nu se oprește pentru anume 3 intrări. De asemeni, observăm din cod că întotdeauna se oprește pentru 0, 1, sau 2. Prin urmare, execuția lui $P(w)$ nu se termină.

5 EQP

Se opresc două programe arbitrare, P_1 și P_2, pe aceleași intrări? (reduceți atât PO, cât și $\overline{PO}$ la EQP)

Obiectiv: $PO <_T EQP$

$$F(P, w) = (Q1, Q2)$$

```
1: function Q1(x):
2:     P(w)
3:     return
4: end function
```

```
1: function Q2(x):
2:     return
3: end function
```

1. $PO(P, w) = 1 \rightarrow EQP(Q1, Q2) = 1$.
   - Dacă apelul $P(w)$ se oprește, ambele programe, Q1 și Q2, se opresc pentru orice intrare.

2. $EQP(Q1, Q2) = 1 \rightarrow PO(P, w) = 1$.

   - Știm că Q1 și Q2 se opresc pe aceleași intrări. De asemeni, știm că Q2 se oprește pentru orice intrare. Prin urmare, pentru ca Q1 să se oprească pentru orice intrare, $P(w)$ trebuie să se termine.

Obiectiv: $\overline{PO} <_T EQP$

$$F(P, w) = (Q1, Q2)$$

```
1: function Q1(x):
2:     P(w)
3:     return
4: end function
```

```
1: function Q2(x):
2:     while(1);
3: end function
```

1. $\overline{PO}(P, w) = 1 \rightarrow EQP(Q1, Q2) = 1$.

   - Dacă apelul $P(w)$ nu se oprește, ambele programe, Q1 și Q2, NU se opresc, indiferent de intrare.

2. $EQP(Q1, Q2) = 1 \rightarrow \overline{PO}(P, w) = 1$.

   - Știm că Q1 și Q2 se opresc pentru aceleași intrări. De asemeni, știm că Q2 NU se oprește indiferent de intrare. Prin urmare, pentru ca Q1 să aibă același comportament, $P(w)$ trebuie să NU se termine.